

1. Introdução

O Mist Models Stylist é um sistema que aproxima clientes de profissionais de consultoria de imagem, moda e estilo. Pelo aplicativo, o cliente consulta estilistas, solicita um atendimento e acompanha o agendamento. O prestador visualiza as solicitações recebidas, aceita ou recusa atendimentos e atualiza seus estados. A solução reúne backend, aplicativo móvel para os dois perfis, banco de dados e mensageria assíncrona.

O backend foi desenvolvido com Node.js, TypeScript, Fastify, Prisma ORM, PostgreSQL e RabbitMQ. O aplicativo foi implementado em Flutter e Dart. Embora o enunciado apresente aplicativo cliente e aplicativo prestador como componentes diferentes, o projeto utiliza uma única aplicação Flutter com fluxos separados conforme o papel autenticado, CLIENTE ou ESTILISTA. Isso reduziu duplicações e permitiu compartilhar autenticação, tema, componentes e serviços HTTP.

A organização foi inspirada em Clean Architecture, separando entrada HTTP, regras de negócio, persistência e infraestrutura. A comunicação principal ocorre por REST, enquanto o RabbitMQ publica e consome eventos relacionados à criação e à alteração de agendamentos.

2. Arquitetura implementada

A solução é composta pelo aplicativo Flutter, pela API REST, pelo PostgreSQL e pelo RabbitMQ. Após o login, o aplicativo identifica o papel do usuário e direciona cada perfil ao seu fluxo.

O cliente pode consultar profissionais, visualizar detalhes, selecionar serviço, data e horário, criar um agendamento e acompanhar seu estado. No perfil de prestador, o dashboard apresenta solicitações pendentes e atendimentos processados. A tela de detalhes permite aceitar ou recusar o serviço, e o acompanhamento das solicitações em andamento ocorre no próprio dashboard.

O backend centraliza autenticação, clientes, estilistas e agendamentos. As rotas são divididas por domínio e seguem a estrutura Routes, Controller, Service e Repository. As rotas definem endpoints e validações; os controladores interpretam requisições e respostas; os serviços concentram regras de negócio; e os repositórios acessam o PostgreSQL por meio do Prisma.

Essa divisão aproxima o projeto dos princípios de Clean Architecture, pois reduz a mistura entre detalhes HTTP, regras e persistência. A implementação não é pura, já que serviços ainda utilizam repositórios concretos e alguns tipos do Prisma

aparecem em camadas superiores. Mesmo assim, a organização melhora a leitura e a manutenção, conforme Martin (2019).

A autenticação utiliza JWT em cookie HTTP-only e bcrypt para proteger senhas. Um middleware restringe o acesso às rotas protegidas. Embora o documento da disciplina indique Express para projetos Node.js, foi utilizado Fastify por seu suporte a plugins, schemas e OpenAPI. A escolha mantém a API REST no ecossistema Node.js, mas representa uma adaptação tecnológica.

No Flutter, o ApiClient concentra requisições, serialização JSON, timeout e tratamento de erros. AuthService, AgendamentoService e EstilistaService organizam as operações por domínio. O gerenciamento de cookies mantém a sessão autenticada, e a separação entre telas, modelos, serviços e widgets evita que a interface concentre toda a lógica.

O modelo de dados contém User, Cliente, Estilista e Agendamento. User armazena autenticação e papel; Cliente e Estilista guardam informações específicas; e Agendamento relaciona os dois perfis, contendo data, serviço, descrição e status. Enums limitam os estados a valores como PENDENTE, ACEITO, RECUSADO, CANCELADO e CONCLUIDO.

A mensageria foi implementada com RabbitMQ. Após criar um agendamento, o backend salva os dados e publica uma mensagem na fila agendamento.prioridade. Quando o status é alterado, publica outra mensagem na fila agendamento.status. Um worker consome as mensagens. As filas são duráveis, as mensagens são persistentes e o consumidor utiliza prefetch, ack, nack e tentativas de reprocessamento.

O fluxo completo começa quando o cliente cria uma solicitação. A API valida os dados, persiste o agendamento e publica o evento. O worker consome a mensagem, enquanto o aplicativo do prestador realiza polling periódico para consultar novas solicitações. Assim, elas aparecem sem atualização manual. Quando o prestador aceita ou recusa, o backend atualiza o banco e publica um evento de mudança de status. O aplicativo do cliente também realiza polling e apresenta automaticamente o novo estado.

O aplicativo não acessa diretamente o RabbitMQ, evitando exposição de credenciais e acoplamento. O backend gerencia o MOM, enquanto o polling leva às interfaces o estado atualizado da API.

3. Decisões de design

REST foi escolhido como interface pública porque representa adequadamente recursos como usuários, estilistas e agendamentos. Os métodos HTTP tornam a API previsível e estão alinhados ao estilo descrito por Fielding (2000).

O Prisma foi adotado para definir o schema, executar migrações e realizar consultas tipadas. Isso reduziu SQL manual, embora tenha criado algum acoplamento com as camadas superiores. Uma evolução possível seria introduzir DTOs e interfaces próprias do domínio.

O RabbitMQ foi aplicado somente aos eventos que justificam processamento assíncrono: criação de agendamento e alteração de status. Cadastros, autenticação e consultas permanecem síncronos. Dessa forma, a EDA complementa o REST sem aumentar desnecessariamente a complexidade.

A manutenção dos dois perfis no mesmo projeto Flutter também foi uma decisão de prazo e reutilização. Os fluxos permanecem separados por telas, serviços e permissões, mas compartilham infraestrutura, componentes e autenticação.

4. Dificuldades e soluções adotadas

A primeira dificuldade foi acessar o backend pelo emulador Android. Como localhost aponta para o próprio emulador, a URL foi configurada como <http://10.0.2.2:3333>. Em dispositivos físicos, deve ser utilizado o IP do computador na rede local.

Outra dificuldade foi preparar PostgreSQL e RabbitMQ de forma reproduzível. A solução foi utilizar Docker Compose, centralizando a configuração e a inicialização dos serviços.

A autenticação com dois perfis exigiu uma estrutura comum de login. User centraliza e-mail, senha, token e papel, enquanto Cliente e Estilista armazenam dados específicos. No Flutter, o armazenamento e o envio de cookies foram isolados para evitar falhas de sessão.

Na mensageria, foi necessário impedir que uma indisponibilidade do RabbitMQ bloqueasse totalmente a operação principal. O banco permanece como fonte de estado, enquanto a publicação é tratada como processamento complementar. Em produção, o padrão Transactional Outbox poderia reduzir o risco de persistir uma alteração sem publicar seu evento.

Também foi necessário equilibrar Clean Architecture e prazo. Foram preservadas as separações essenciais, mas sem criar abstrações que aumentariam excessivamente a complexidade de um projeto acadêmico.

5. Reflexão sobre os padrões estudados

REST foi adequado para entrada, consulta e atualização de recursos, além de facilitar testes com Postman e documentação com Swagger e Scalar. Como não envia sozinho atualizações futuras aos aplicativos, foi complementado por mensageria e polling.

Clean Architecture orientou a separação entre regras de negócio, protocolo HTTP e persistência. Mesmo aplicada de forma parcial, tornou o sistema mais organizado e simples de evoluir.

A Event-Driven Architecture foi aplicada ao representar a criação e a alteração de agendamentos como eventos. O produtor publica a ocorrência sem conhecer todos os futuros consumidores. Isso reduz acoplamento e permite adicionar notificações, auditoria ou métricas, mas também introduz consistência eventual e necessidade de monitoramento, como discute Richardson (2018).

O MOM foi representado pelo RabbitMQ, que realiza comunicação indireta entre produtores e consumidores. Filas duráveis, persistência, confirmação e controle de consumo aumentam a confiabilidade, seguindo padrões apresentados por Hohpe e Woolf (2003).

Na Sprint 4, foi concluído o fluxo do prestador e consolidada a integração de ponta a ponta. O cliente cria a solicitação, o backend persiste os dados e publica um evento, o worker consome a mensagem, o prestador identifica a solicitação pelo polling, aceita ou recusa e o cliente recebe o novo estado em uma consulta automática posterior. O polling atende ao requisito de atualização sem ação manual, embora gere requisições mesmo quando não existem mudanças.

A integração foi validada com testes manuais da API, execução dos dois perfis e acompanhamento do PostgreSQL e do RabbitMQ no ambiente Docker. O resultado demonstra a comunicação entre Flutter, REST, banco de dados, MOM e worker.

6. Considerações finais

O Mist Models Stylist integra cliente, prestador, backend, banco de dados e mensageria em uma solução funcional. O sistema permite cadastro, autenticação, consulta de estilistas, criação de agendamentos, aceite ou recusa e acompanhamento automático de status.

REST organizou a comunicação principal; Clean Architecture orientou a divisão interna; EDA representou acontecimentos relevantes como eventos; RabbitMQ forneceu a infraestrutura de mensagens; e o polling atualizou as interfaces sem recarregamento manual.

Como melhorias futuras, podem ser implementados WebSockets ou notificações push no lugar do polling, testes automatizados, Dead Letter Queues, maior observabilidade dos workers e menor acoplamento com tipos do Prisma. Mesmo com essas possibilidades, a versão atual apresenta uma arquitetura coerente e alinhada aos conceitos estudados.

Referências:

COULOURIS, George et al. Distributed Systems: Concepts and Design. 5. ed. Boston: Addison-Wesley, 2011.

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Tese de doutorado. University of California, Irvine, 2000.

HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Boston: Addison-Wesley, 2003.

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

RICHARDSON, Chris. Microservices Patterns: With Examples in Java. Shelter Island: Manning, 2018.